Lecture 04

Let expressions, Local Functions, Options

OCaml So Far

- Types

  `int` `bool` `t1 -> t2` `t1 * t2 * … * tn` `t list`

- Expressions

  `34` `true` `(e1,e2,…,en)` `[]` `e1::e2` `x`

  `f(e1,e2,…,en)` `e1 + e2` `if e1 then e2 else e3`

  `fst` `snd` `e = []` `List.hd` `List.tl`

- Bindings

  ```
  let x = e
  let [rec] f ((x1:t1), … (xN:tN)) = e
  ```

OCaml So Far

- Types


```
int   bool   t1 -> t2   t1 * t2 * … * tn   t list
```

- Expressions


```
34   true   (e1,e2,…,en)   []   e1::e2   x
f(e1,e2,…,en)   e1 + e2   if e1 then e2 else e3
fst   snd   e = []   List.hd   List.tl
```

- Bindings

```
let x = e
let [rec] f ((x1:t1), … (xN:tN)) = e
```

OCaml So Far

- Types

  can nest

```
int   bool   t1 -> t2   t1 * t2 * … * tn   t list
```

- Expressions

  can nest

```
34   true   (e1,e2,…,en)   []   e1::e2   x
f(e1,e2,…,en)   e1 + e2   if e1 then e2 else e3
fst   snd   e = []   List.hd   List.tl
```

- Bindings

  ???

```
let x = e
let [rec] f ((x1:t1), … (xN:tN)) = e
```

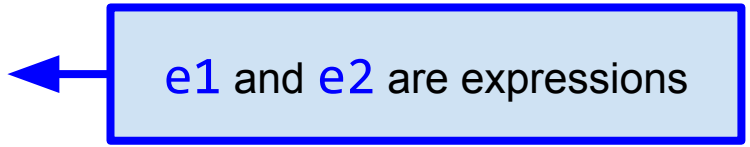
Local Bindings aka *Let Expressions*

Expressions can have local (private) variable and function bindings

- For style and convenience: lets us name intermediate results
- For consistency: everything else can nest
- For efficiency: avoid unnecessary recomputation
  - *Not* “just a little faster”
- For safety and maintenance: hide implementation details from clients

Let Expressions : Local Variable Binding

Syntax: `let x = e1 in e2`



Let Expressions : Local Variable Binding

Syntax: `let x = e1 in e2`

Type Checking

- If `e1` has type `t1` in current static environment AND
- If `e2` has type `t2` in static environment extended with `x` ↦ `t1`
- Then `let x = e1 in e2` has type `t2`
- Else, report error and fail

Let Expressions : Local Variable Binding

Syntax: `let x = e1 in e2`

Evaluation

- If `e1` evaluates to `v1` in current dynamic environment AND
- If `e2` evaluates to `v2` in dynamic environment extended with `x` ↦ `v1`
- Then `let x = e1 in e2` evaluates to value `v2`

Let Expressions are “Just Expressions”!

A let expression can go anywhere an expression can go

*Because a let expression **is an expression***

Also: “Multiple bindings” is just nested let-expressions

- But as style, don’t indent them that way

Local Bindings: What's New Here?

- Scope
  - More control over which parts of a program can access a binding
  - For **`let`** expressions: *just the body of the let* !
- Nothing else is really new
  - **`let`** expressions just work pretty much like top-level **`let`** bindings

Let Expressions : Local Function Binding

Syntax: `let [rec] f ((x1:t1), ..., (xN:tN)) = e in e2`

Type Checking: Same as top-level function binding

- Using static environment where the let expression occurs

Evaluation rules: Same as top-level function binding

- Using dynamic environment where the let expression occurs

A local binding: `f` used only in `e` (if rec present) and `e2` (always)

Let Expression Example

```
nats_upto 5;;
- : int list = [0; 1; 2; 3; 4]
```

Not great style:

```
let nats_upto (x : int) =
  let rec range ((lo : int),(hi : int)) =
    if lo < hi then
      lo :: range (lo + 1,hi)
    else
      []
  in
  range (0,x)
```

range is hidden…

No one else can use it!

Also, does **range**

really need the **hi** parameter?

Let Expression Example

Better style:

```
let nats_upto (x : int) =
  let rec loop (lo : int) =
    if lo < x then
      lo :: loop (lo + 1)
    else
      []
  in
  loop 0
```

```
nats_upto 5;;
- : int list = [0; 1; 2; 3; 4]
```

- Functions can use bindings from the environment where they are defined. Including:
  - “outer” environments
  - parameters to outer function
  - etc.
- Unnecessary params are *bad style*

Nested Functions: Style

- Good style to define helper functions inside if they are:
  - Not useful elsewhere (avoid clutter)
  - Likely to be misused elsewhere (improve reliability)
  - Likely to be changed or removed (hide implementation detail)
- Fundamental tradeoff:
  - Reusing code saves efforts and (usually) avoids bugs, BUT
  - Makes it harder to change later (lots of folks depend on its details)

DANGER: Avoid Repeated Recursion ⚠️

- How much work does this function do?

```
let rec bad_max (xs : int list) =
 if xs = [] then
   0 (* bad style, will fix later *)
 else if List.tl xs = [] then
   List.hd xs
 else if List.hd xs > bad_max (List.tl xs) then
   List.hd xs
 else
   bad_max (List.tl xs)
```

DANGER: Avoid Repeated Recursion ⚠️

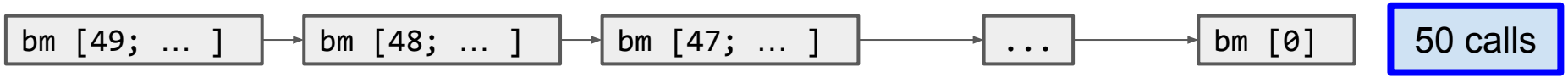
- How much work does this function do?

```
bad_max (List.rev (nats_upto 50));;
- : int = 49 (* returns in microseconds *)

bad_max (nats_upto 50);;
- : int = ... (* still running *)
```

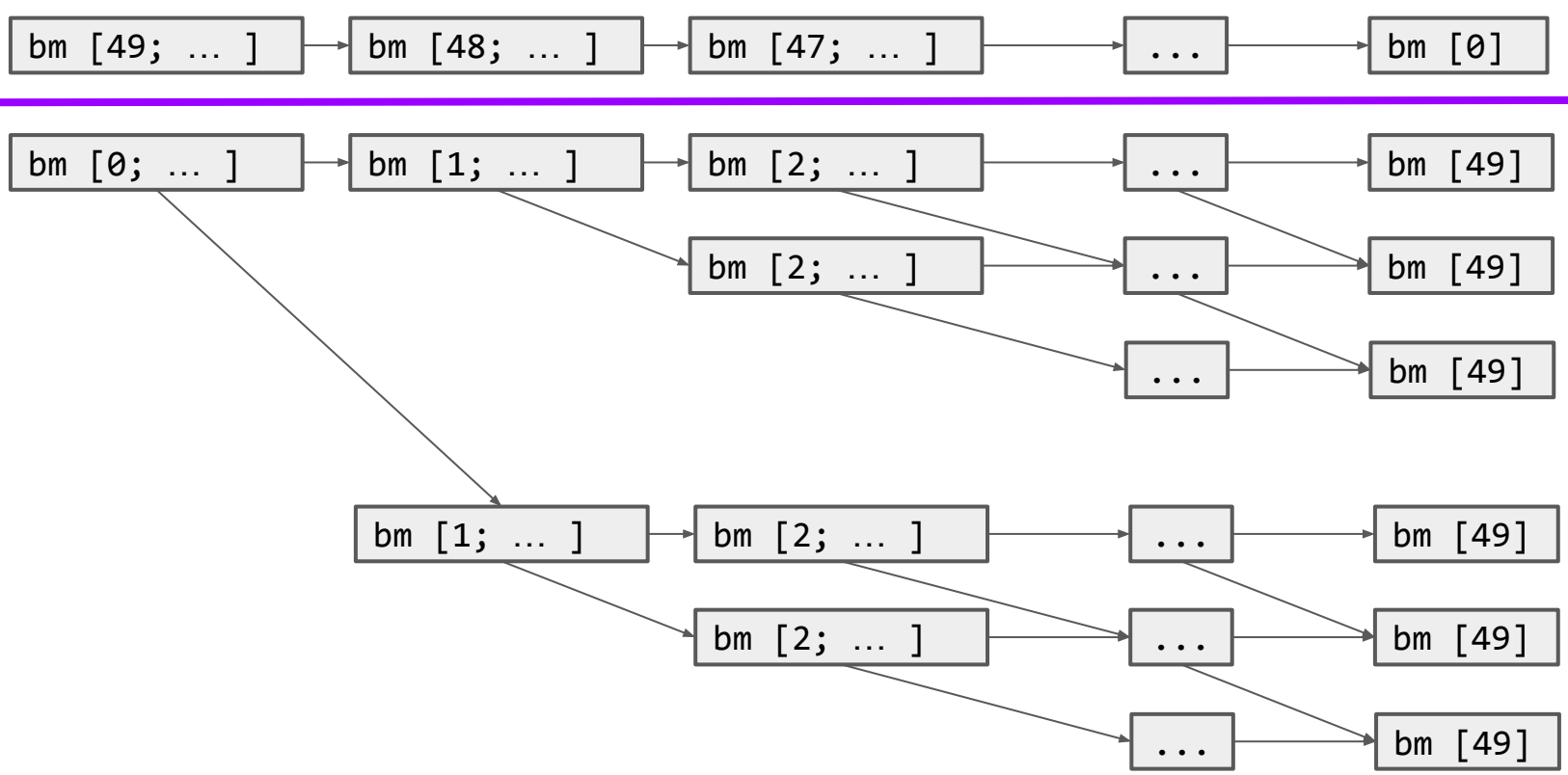
Fast vs. Unusable

```
if List.hd xs > bad_max (List.tl xs)
then List.hd xs
else bad_max (List.tl xs)
```



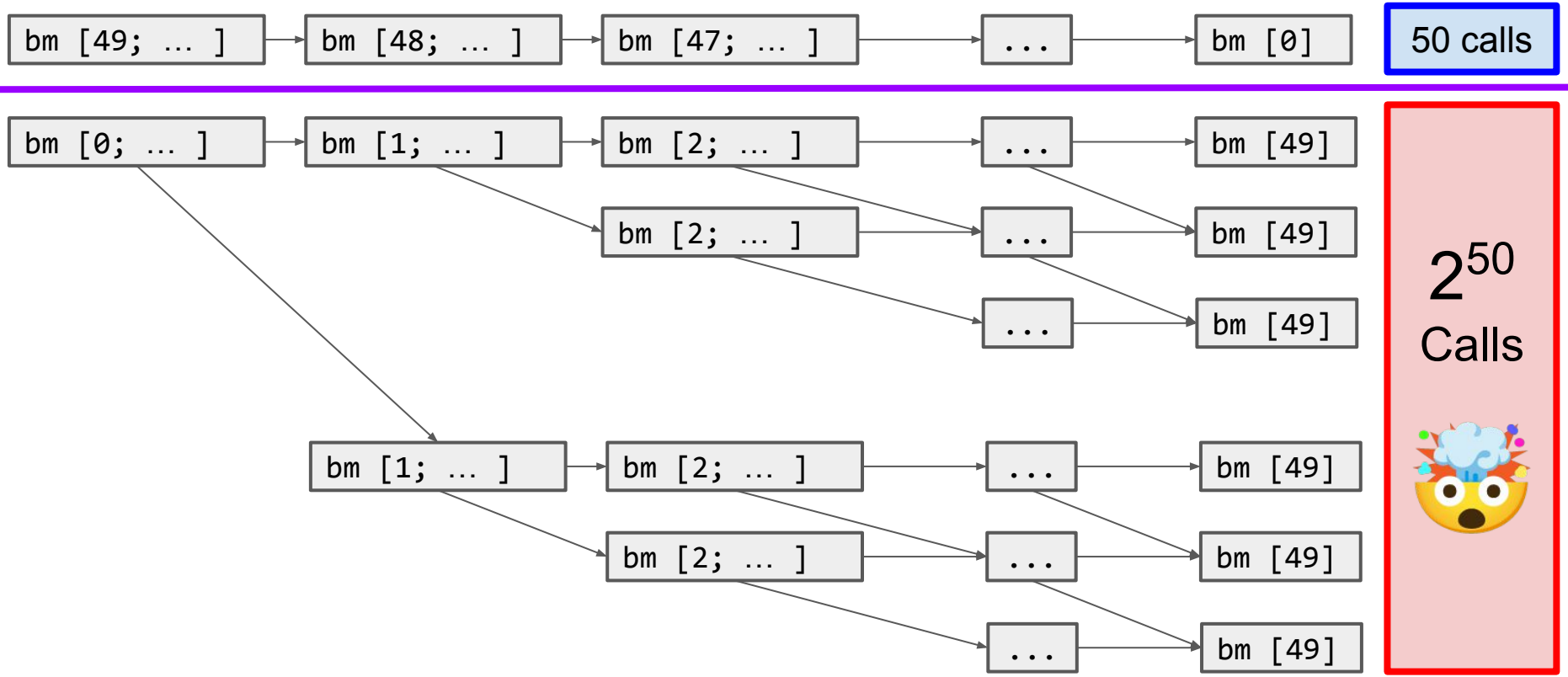
Fast vs. Unusable

```
if List.hd xs > bad_max (List.tl xs)
then List.hd xs
else bad_max (List.tl xs)
```



Fast vs. Unusable

```
if List.hd xs > bad_max (List.tl xs)
then List.hd xs
else bad_max (List.tl xs)
```



Math and explosion

- Suppose the logic in one `bad_max` call *not counting recursion* takes 1 microsecond
  - It doesn't matter if this guess is off by 1000x
- Then `bad_max [49; ... ; 0]` takes 50 microseconds
- And `bad_max [0; ... ; 49]` takes 2^{50} microseconds
  - That's 35.7 years
  - And `bad_max [0; ... ; 51]` takes 4x longer

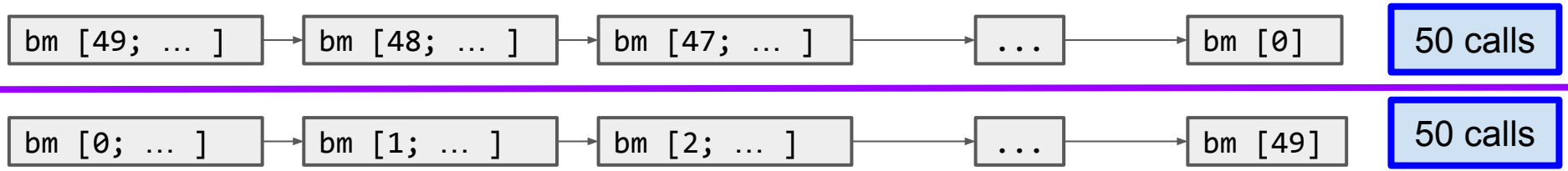
Using `let` to Avoid Repeated Recursion

```
let rec better_max (xs : int list) =
 if xs = [] then
   0 (* bad style *)
 else if List.tl xs = [] then
   List.hd xs
 else
   let tl_max = better_max (List.tl xs) in
   if List.hd xs > tl_max then
     List.hd xs
   else
     tl_max
```

- Recurse over the list tail *once*
- Store *result* in environment, bound to **`tl_max`**

Fast vs. Fast

```
let tl_max = better_max (List.tl xs) in
   if List.hd xs > tl_max then
     List.hd xs
   else
     tl_max
```



Data?

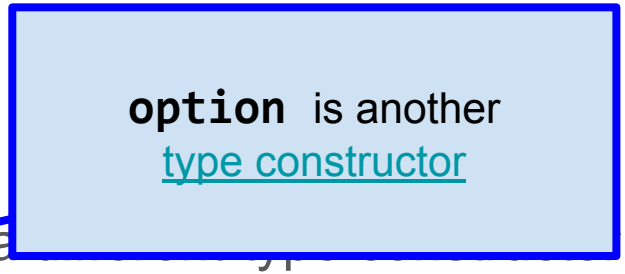
What to do about partial functions?

- Many computations are not defined or “misbehave” for some inputs
  - `List.hd []`, `List.tl []`, maximum element of `[]`, 1 / 0, etc.
- OCaml does provide *exceptions* for such cases…
  - We will see and use exceptions more later
  - But exceptions let you forget-about-the-exceptional-thing
  - And that can lead to surprising and undesirable control flow
- OCaml also provides an `option` type for handling partial functions
  - For when your data structure “may or may not have some data”

Options

- Basically: “lists” of length zero or one, but a *different* type constructor
- A value of type `t option` can either be:
  - `None`
    - Valid value of type `t option` for *any* type `t`
      - Like `[]` for `t list`
  - `Some v`
    - Where value `v` has type `t`
      - Like `[v]` for `t list`

Options



- Basically: “lists” of length zero or one, but a
- A value of type `t option` can either be:
  - `None`
    - Valid value of type `t option` for *any* type `t`
      - Like `[]` for `t list`
  - `Some v`
    - Where value `v` has type `t`
      - Like `[v]` for `t list`

Options

- Basically: “lists” of length zero or one, but a
- A value of type `t option` can either be:
  - `None`
    - Valid value of type `t option` for *any* t
      - Like `[]` for `t list`
  - `Some v`
    - Where value `v` has type `t`
      - Like `[v]` for `t list`

`option` is another type constructor

Just like our other friends:

`t1 * t2`

`t1 -> t2`

`t list`

Options : Build -- None

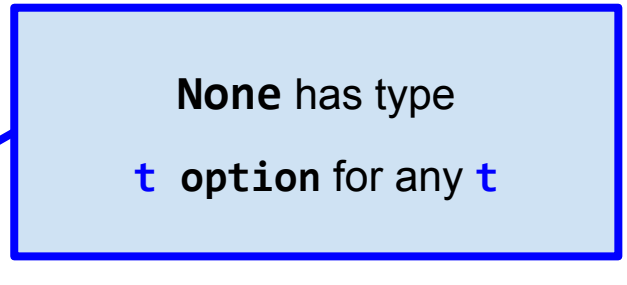
Syntax: `None`

Type Checking

- `None` has type `'a option`

Evaluation

- `None` is a value



Options : Build -- Some

Syntax: `Some e`

- Where `e` is an expression

Type Checking

- If `e` has type `t`
- Then `Some e` has type `t option`
- Else report error and fail

Evaluation

- If `e` evaluates to value `v`
- Then `Some e` evaluates to value `Some v`

Options : Use -- Test for None

Syntax: `e = None`
- Where `e` is an expression

Type Checking
- If `e` has type `t option`
- Then `e = None` has type `bool`
- Else report error and fail

Evaluation
- If `e` evaluates to value `None`,
- Then `e = None` evaluates to value `true`
- Else `e = None` evaluates to value `false`

Alternately, use library functions **`Option.is_some`** or **`Option.is_none`**

Options : Use -- Get the element

Syntax: `Option.get e`

- Where `e` is an expression

Type Checking

- If has `e` has type `t option`
- Then `Option.get e` has type `t`
- Else report error and fail

Evaluation

- If `e` evaluates to value `Some v`
- Then `Option.get e` evaluates to value `v`
- Else `e` evaluated to `None` so `Option.get e` raises an exception

A Better Better Better Max

```
let rec good_max (xs : int list) =
  if xs = [] then
    None
  else
    let tl_ans = good_max (List.tl xs) in
    if not (tl_ans = None) && Option.get tl_ans > List.hd xs then
      tl_ans
    else
      Some (List.hd xs)
```

A Better Better Better Max

```
let rec good_max (xs : int list) =
  if xs = [] then
    None
  else
    let tl_ans = good_max (List.tl xs) in
    if not (tl_ans = None) && Option.get tl_ans > List.hd xs then
      tl_ans
    else
      Some (List.hd xs)
```

returns an `int option`, *NOT* an `int`, so can finally give a reasonable answer for even "bad" inputs (i.e., `[]`)

A Better Better Better Max

```
let rec good_max (xs : int list) =
  if xs = [] then
    None
  else
    let tl_ans = good_max (List.tl xs) in
    if not (tl_ans = None) && Option.get tl_ans > List.hd xs then
      tl_ans
    else
      Some (List.hd xs)
```

clients are forced to consider data and no-data cases

Group Work

1. For each program point labeled 1, 2, 3, 4, write the static and dynamic environments at that point

```
let a = 1 (* 1 *)
let b = 2 (* 2 *)
let c = let a = 3 in let c = 4 in (* 3 *) (a + b) * c
(* 4 *)
```

2. Write a function `f` that takes an `int option list` and returns an `int list` consisting of the non-None elements of the input list incremented by 1.
   - Example: `f [None; Some 1; None; Some 5] = [2; 6]`

Aliasing and Mutation

Can you spot the difference? Can a client?

```
let sort_pair (pr : int*int)=
   if fst pr < snd pr then
     pr
   else
     (snd pr, fst pr)
```

```
let sort_pair (pr : int*int)=
   if fst pr < snd pr then
     (fst pr, snd pr)
   else
     (snd pr, fst pr)
```

Can you spot the difference? Can a client?

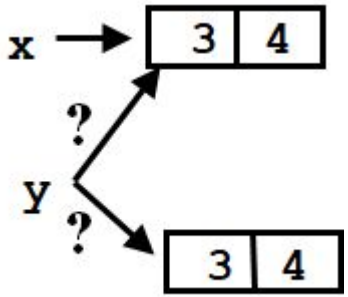
```
let sort_pair (pr : int*int)=
   if fst pr < snd pr then
     pr
   else
     (snd pr, fst pr)
```

```
let sort_pair (pr : int*int)=
   if fst pr < snd pr then
     (fst pr, snd pr)
   else
     (snd pr, fst pr)
```

- If pair already sorted, does this return:
  - (an alias to) the same pair, or
  - a new pair with the same contents?
- In OCaml, *we cannot tell* which way `sort_pair` is implemented (!!)
  - This "*negative expressiveness*" is a big deal!
  - When data is immutable, aliasing does not matter

Why lack-of-mutation matters

```
let x = (3,4)
let y = sort_pair x
somehow mutate fst x to hold 5
let z = fst y (* 3 or 5 ?? *)
```

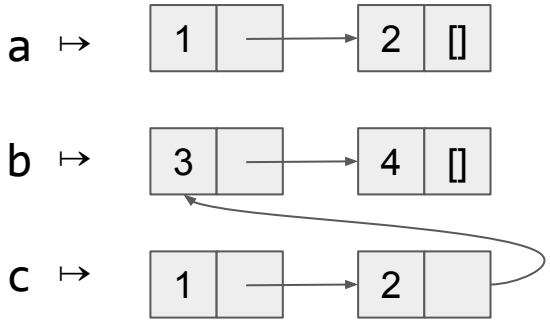


- What is `z`?
  - Would depend on how we implemented `sort_pair`
- Would have to decide carefully and document `sort_pair`
  - Without mutation, we can implement "either way"
    - No code can ever distinguish aliasing vs. identical copies
    - No need to think about aliasing: focus on other things
    - Can use aliasing, which saves space, without danger

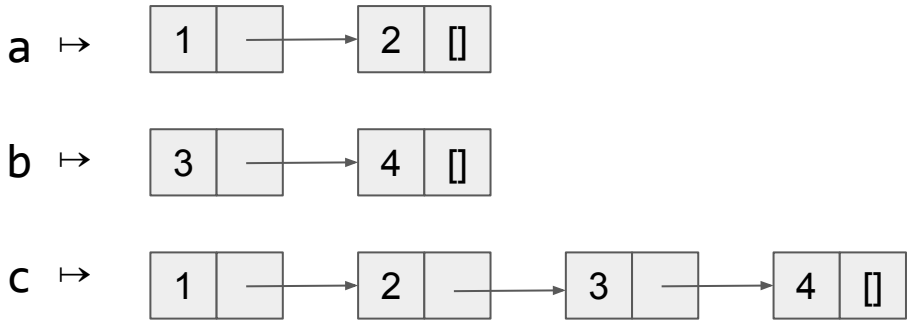
Aliasing with append

```
let rec append ((xs:'a list),(ys:'a list)) =
   if xs = []
   then ys
   else List.hd xs :: append (List.tl xs, ys)
let a = [1; 2]
let b = [3; 4]
let c = append (a,b)
```

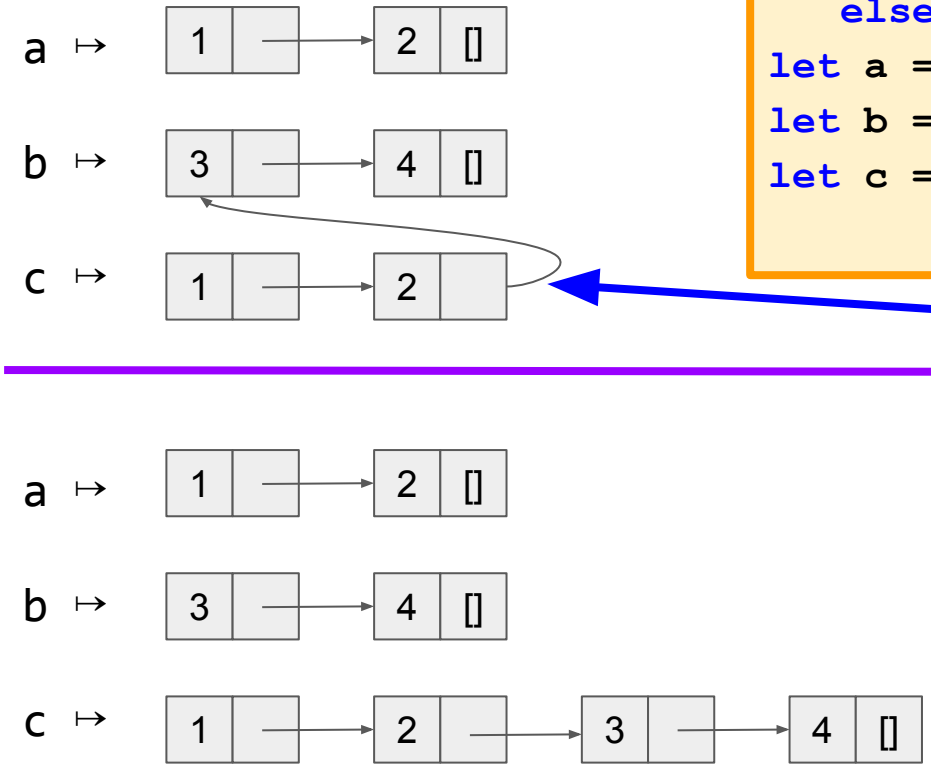
Aliasing with append




```
let rec append ((xs:'a list),(ys:'a list)) =
   if xs = []
   then ys
   else List.hd xs :: append (List.tl xs, ys)
let a = [1; 2]
let b = [3; 4]
let c = append (a,b)
```



Aliasing with append




```
let rec append ((xs:'a list),(ys:'a list)) =
   if xs = []
   then ys
   else List.hd xs :: append (List.tl xs, ys)
let a = [1; 2]
let b = [3; 4]
let c = append (a,b)
```

Clients can never tell!

(but it's actually the upper one)

Can *safely* reuse and share data.

Immutability can be *more efficient*!

Immutability is great!

- In OCaml, we create aliases all the time without thinking about it
  - It is *impossible* to tell where there is aliasing
  - Example: `List.tl` is constant time; doesn't copy anything
  - No need to worry!
- In languages with mostly mutable data (e.g., Java), programmers are *obsessed* with aliasing and object identity
  - They have to be (!)
  - So that later assignments affect the right parts of the program
  - Often crucial to make copies in just the right places
    - Consider a Java example

Java security nightmare (bad code)

```
class ProtectedResource {
   private Resource theResource = ...;
   private String[] allowedUsers = ...;
   public String[] getAllowedUsers() {
     return allowedUsers;
   }
   public String currentUser() { ... }
   public void useTheResource() {
     for(int i=0; i < allowedUsers.length; i++) {
         if(currentUser().equals(allowedUsers[i])) {
             ... // access allowed: use it
             return;
         }
     }
     throw new IllegalAccessException();
   }
}
```

Have to make copies

The problem:*

```
p.getAllowedUsers()[0] = p.currentUser();
p.useTheResource();
```

The fix:

```
public String[] getAllowedUsers() {
    … return a copy of allowedUsers …
}
```

Copying immutable data is unnecessary!

*CSE 331 calls this “representation exposure”

= versus ==

Equality in OCaml

- OCaml has two different equality operators: `=` and `==`
  - `e1 <> e2` is the same as `not (e1 = e2)`
  - `e1 != e2` is the same as `not (e1 == e2)`
- For CSE 341 always use `=` (or `<>`)
  - *Not* the character sequences you are used to ⚠️
  - It turns out not to matter for ints and bools though
- The difference between them is interesting and relevant to aliasing...

Structural equality

- `=` is defined recursively: same data in same shape
- `3=3`, `(1,2)=(1,2)`, `[3;4;5]=[3;4;5]`, `None=None`, `Some 7 = Some 7`, etc.
- Two subexpressions must have same type
  - Hard to be equal otherwise anyway
- Notice:
  - If x and y are aliases, then `x=y` must evaluate to true
  - If x and y are not aliases, then `x=y` might evaluate to true or false
  - Aliasing doesn't matter
- More like Java's `equals`, but predefined by the language

Reference equality

- `e1 == e2` is true if `e1` and `e2` evaluate to aliases (or same `int`, `bool`, etc.)
  - For data structures, this is faster to check than structural equality
- But wait: then clients *can* tell if two things are aliases
  - Could complicate writing code and implementing the language
  - And you really shouldn't care about aliases for immutable data
- So OCaml reached a pragmatic compromise (hack?)
  - The "official definition" of == promises almost nothing for immutable data:
    - *all* it guarantees is == implies =
  - If you assume it means more, you assume the risk of surprise
  - Keeps "aliases" out of the language definition
    - a key advantage of immutable data!
- Again: do not use == (nor !=) in CSE 341